

Pricing Intelligence Platform

Software Design Document, Technical Design Document, Architecture Document & Developer Guide

Prepared by: AI Documentation Analysis (Claude)

Generated: July 19, 2026 · Version 1.0 (Reverse-Engineered from Source)

Table of Contents

0.1 Confidentiality Statement	4
1 Executive Summary	5
1.1 Project Overview	5
1.2 Business Objective	5
1.3 Purpose & Vision	5
1.4 Target Users	5
1.5 Business Value & Expected Benefits	5
2 Project Overview (Detailed)	7
2.1 Problem Solved	7
2.2 Industry / Business Domain	7
2.3 Pricing Context	7
2.4 Decision Intelligence Context	7
2.5 AI Capabilities (Summary — see full AI chapter)	7
2.6 Enterprise Value	7
3 High-Level Architecture	8
3.1 Deployment Model	8
3.2 Layered Architecture	8
3.3 Request / Response Flow (typical scored module)	9
3.4 Decision Workflow (Price Engine tab, full pipeline)	9
4 Technology Stack	10
5 Project Structure	12
6 Module Documentation	13
6.1 Module: Overview (Tab 1)	13
6.2 Module: Executive Decision Center (Tab 2)	13
6.3 Module: Price Engine (Tab 3, Python variable tab3)	14
6.4 Module: Simulation Hub (Tab 4)	15
6.5 Module: Digital Twin (Tab 5)	15
6.6 Module: AI Advisor (Tab 6)	16
6.7 Module: Contract Analyzer (Tab 7)	16
6.8 Module: Competitor Pricing (Tab 8)	16
7 Detailed Function Documentation	18
7.1 UI / Formatting Helpers	18
7.2 Legacy Deterministic Rule Functions (superseded by AI, retained in source) . . .	18

7.3	Cortex AI Low-Level Wrappers	19
7.4	AI Memory Layer	21
7.5	Central AI Reasoning Service	21
7.6	PSI / Capacity / Simulation Math	22
7.7	Competitor Tab Local Helpers (defined inside the with tab8: block)	22
8	Database Documentation	24
9	SQL Documentation	27
10	AI Documentation	29
10.1	AI_COMPLETE	29
10.2	AI_CLASSIFY	29
10.3	AI_FILTER	29
10.4	AI_AGG	29
10.5	SNOWFLAKE.CORTEX.SUMMARIZE	29
10.6	AI Memory	29
10.7	Caching	29
10.8	Reasoning Engine / Decision Engine	30
10.9	Where Every AI Function Is Used (Cross-Reference)	30
11	User Interface Documentation	31
11.1	Global Chrome	31
11.2	Per-Tab Widget Inventory	31
11.3	Session-State Persistence Pattern	32
12	Business Logic Documentation	33
12.1	Pricing Formulas (Deterministic — never AI-generated)	33
12.2	Decision Logic (AI-Driven — judgment, not arithmetic)	33
12.3	Alert Generation Logic	34
13	Workflow Documentation	35
13.1	User Workflow (typical session)	35
13.2	AI Workflow (every scored call)	35
13.3	Database Workflow	35
14	Security	36
14.1	Authentication & Authorization	36
14.2	Data Validation	36
14.3	SQL Injection Prevention	36
14.4	Caching & Session State	36
14.5	AI Safety	36
15	Performance	37
15.1	Caching	37
15.2	Database Calls	37
15.3	Batching (AI)	37
15.4	Scalability / Concurrency / Expected Load	37
15.5	Performance Recommendations	37
16	Error Handling	38
17	Testing	39
17.1	Recommended Test Cases (not currently implemented)	39
18	Deployment Guide	40

18.1 Prerequisites 40

18.2 Environment Setup 40

18.3 Configuration 40

18.4 Execution / Production Deployment 40

19 Maintenance Guide 41

19.1 Coding Standards Observed 41

19.2 Extension Guide 41

19.3 Best Practices Going Forward 41

20 Known Limitations 42

21 Conclusion 43

22 Appendix 44

22.1 Glossary 44

22.2 Abbreviations 44

22.3 References 44

0.1 Confidentiality Statement

This document was generated by analyzing the source code of a single Streamlit-in-Snowflake application file (`streamlit_app.py`, the “Pricing Intelligence Platform”). It is intended for internal technical stakeholders — developers, architects, QA engineers, and technical reviewers.

Important scope note: Anthropic’s Claude was given exactly **one** source file. No other modules, configuration files, SQL DDL scripts, requirements files, or infrastructure-as-code were supplied. Every statement in this document is derived strictly from what is observable in that one file. Where the file references a Snowflake table but does not define its schema (because DDL lives outside the application code), the columns are inferred from the SELECT/INSERT statements that touch them, and this is explicitly marked **[Inferred from usage]**. Nothing in this document describes functionality that does not have a corresponding line of code.

This is treated as an accurate, appropriately-scoped Technical Design Document for the artifact actually provided, rather than a padded document simulating a multi-repository enterprise system that was never supplied.

1 Executive Summary

1.1 Project Overview

The **Pricing Intelligence Platform** is a single-page Streamlit application designed to run as a **Streamlit-in-Snowflake (SiS)** app inside a Snowflake account. It provides discrete-manufacturing pricing teams with a consolidated workspace covering cost-to-price calculation, demand/price-sensitivity modelling, capacity-based pricing adjustments, win-probability prediction, discount optimization, “digital twin” market-shock simulation, contract risk analysis, competitor benchmarking, and an AI-generated executive narrative layer built on **Snowflake Cortex AI** functions.

1.2 Business Objective

Discrete manufacturers set prices across many SKUs, customers, and plants under moving input costs (steel, fuel) and variable plant utilization. The application’s stated objective is to replace ad-hoc spreadsheet pricing with a single governed workspace that (a) computes deterministic, auditable cost/price/margin arithmetic, and (b) augments that arithmetic with Cortex-generated business narratives, risk classifications, and recommended actions, so pricing/deal-desk staff and executives can see both the number and the “why” in one place.

1.3 Purpose & Vision

The purpose, as expressed by the code’s structure and inline comments, is to progressively replace **hardcoded business-rule thresholds** (fixed margin tables, fixed win-probability formulas, fixed risk cut-offs) with **Cortex AI reasoning**, while keeping the actual financial arithmetic (cost, revenue, profit multiplication) deterministic and auditable. Numerous in-code comments describe this as a “v2” migration (“no manual weights... AI_COMPLETE is instructed to reason over the full context and return the score itself”).

1.4 Target Users

Based on the language used throughout the UI (labels such as “Executive Decision Center”, “CEO / CFO level summary”, “deal desk manager”, “pricing manager”, “operations executive”), the target users are:

- Pricing analysts / deal-desk managers running per-customer, per-SKU pricing scenarios.
- Sales / commercial managers assessing win probability and discount strategy.
- Plant / operations managers reviewing capacity-driven pricing modifiers.
- Executives (CEO/CFO-level) consuming the Executive Decision Center and AI Advisor tabs.

1.5 Business Value & Expected Benefits

- Consolidates cost, pricing, capacity, discount, contract, and competitor data into one workspace instead of scattered spreadsheets.
- Adds a narrative/explanation layer (via Cortex AI) on top of every score, so a number is always accompanied by a plain-language justification and recommended action.
- Retains deterministic math for anything that is a real financial calculation (revenue, cost, margin), while delegating **judgment-based** decisions (risk severity, market position labels, discount justification, executive narrative) to Cortex AI — an explicit, repeated design principle in the source comments.

- Defensive engineering: every Cortex call is wrapped in try/except, cached, and has an explicit fallback or “AI Unavailable” state, so a Cortex outage degrades the UI gracefully rather than crashing it.

2 Project Overview (Detailed)

2.1 Problem Solved

Manufacturing pricing must reconcile: standard cost (raw materials + surcharges), target/ceiling/floor price bands, customer-specific negotiated discounts, plant capacity pressure, competitor market pricing, and deal win-probability — all while producing an auditable, explainable recommendation. The application solves this by chaining deterministic price adjustments (Cost → PSI margin → Capacity modifier → Customer discount → Final price) and by attaching an AI-generated recommendation/justification at each stage.

2.2 Industry / Business Domain

Discrete manufacturing (the code refers to “steel surcharge,” “fuel surcharge,” “plant capacity,” and “manufacturing pricing strategist” in AI prompts), operating a B2B sales model with negotiated customer agreements and multiple named competitors.

2.3 Pricing Context

Price Sensitivity Index (PSI) per customer, dynamic surcharges, capacity utilization modifiers, and customer-specific discounts are the primary levers modelled.

2.4 Decision Intelligence Context

The application implements a **central AI reasoning pipeline** (`ai_reason_score`) that is reused across every scoring module (Executive Health, PSI margin, Capacity modifier, Win Probability, Contract Risk, Margin Leakage, Advisor Health, Competitor Position, Discount Scenario Selection). This is the architectural core of the “decision intelligence” story: one reusable service, many callers.

2.5 AI Capabilities (Summary — see full AI chapter)

`AI_COMPLETE`, `AI_CLASSIFY`, `AI_FILTER`, `SNOWFLAKE.CORTEX.SUMMARIZE`, and `AI_AGG` (via a `VALUES` table constructed in SQL) are all used. An in-session (plus best-effort persisted) **AI memory** log gives the reasoning pipeline historical context per module/decision type.

2.6 Enterprise Value

Single governed pricing surface, defensive fallback behavior on AI outage, a diagnostics panel for operators to self-service Cortex connectivity issues, and a consistent reusable scoring/rendering pattern (`ai_reason_score` + `render_ai_score_card`) instead of bespoke logic per module.

3 High-Level Architecture

3.1 Deployment Model

The file's imports (from `snowflake.snowpark.context import get_active_session`) and its use of `st.set_page_config` plus native Streamlit widgets indicate this is built to run as a **Streamlit-in-Snowflake (SiS)** application: Streamlit executes inside Snowflake's runtime, and `get_active_session()` retrieves the Snowpark session automatically (no manual connection string, credentials, or `st.secrets` are present anywhere in the file). All data access and all AI calls are issued as Snowflake SQL (`session.sql(...).collect() / .to_pandas()`), never via an external REST client.

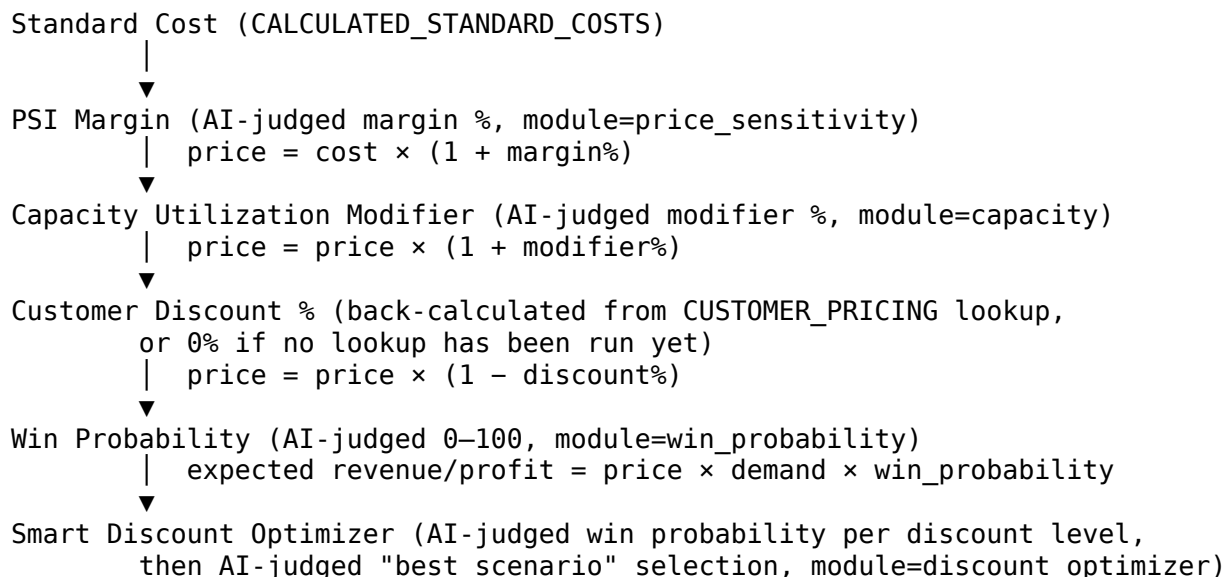
3.2 Layered Architecture

Presentation Layer (Streamlit) - Page config, global CSS (glassmorphism dark theme) - 8 tabs: Overview, Executive Decision Center, Price Engine, Simulation Hub, Digital Twin, AI Advisor, Contract Analyzer, Competitor Pricing - Reusable UI helpers: <code>section()</code>, <code>divider()</code>, <code>health_bar_html()</code>
Session State Layer - <code>_init_state()</code> guards ~14 persisted keys so widget reruns do not wipe computed results (<code>opt_results</code>, <code>sim_results</code>, <code>dt_results</code>, ...) - <code>ai_cache</code> dict memoizes every Cortex call within the session
Business Logic Layer (deterministic) - <code>safe_pct_change</code>, <code>get_psi_margin/get_psi_score</code> (legacy tables), <code>get_capacity_modifier</code> (legacy), <code>compute_win_probability</code> (fast estimator for chart sweeps), digital-twin cost/demand math, contract margin math, price-optimization elasticity sweep
AI Reasoning Layer (Cortex) - Low-level wrappers: <code>ai_complete</code> / <code>ai_classify</code> / <code>ai_filter</code> / <code>ai_summarize</code> / <code>ai_agg_over_rows</code> - <code>parse_ai_json</code>: lenient JSON extraction from LLM text - AI Memory: <code>ai_memory_log</code> / <code>get_ai_memory_context</code> - Central pipeline: <code>ai_reason_score</code> / <code>ai_reason_score_batch</code> - Renderer: <code>render_ai_score_card</code>
Data Access Layer (Snowpark) - <code>session.sql(...).to_pandas() / .collect()</code> against <code>PRICING_ENGINE_DB.CORE_INPUT</code> / <code>CORE_OUTPUT</code> / <code>CORE_INTERNAL</code>
Snowflake Platform - Base tables (cost, agreements, capacity, competitor pricing) - Output tables written by the app (simulation, optimization, digital twin, AI memory log) - Cortex AI functions (<code>AI_COMPLETE</code>, <code>AI_CLASSIFY</code>, <code>AI_FILTER</code>, <code>AI_AGG</code>, <code>SNOWFLAKE.CORTEX.SUMMARIZE</code>)

3.3 Request / Response Flow (typical scored module)

1. User selects inputs via Streamlit widgets (SKU, customer, plant, sliders).
2. App loads deterministic facts from Snowflake (`_safe_load / direct session.sql`).
3. Deterministic arithmetic (cost $\times$ margin, revenue, profit) is computed in Python.
4. `ai_reason_score(module, decision_type, context_facts, ...)` is called:
 - a. Build a cache key from module + decision type + facts.
 - b. If cached, return immediately (no network round-trip).
 - c. Else, retrieve prior in-session/Snowflake memory for that module via `get_ai_memory_context`.
 - d. Compose a prompt instructing the model to return a strict JSON object.
 - e. Call `ai_complete()` $\rightarrow$ issues `SELECT AI_COMPLETE(model, prompt)` to Snowflake.
 - f. `parse_ai_json()` leniently extracts JSON (handles fences, extra prose, trailing commas).
 - g. `_coerce_score_payload()` validates/normalizes the payload; only a genuinely un-parseable score fails the attempt.
 - h. If invalid, **one repair retry** is issued with a stricter “JSON only” instruction.
 - i. If still invalid, a clearly labeled AI Unavailable neutral payload is returned — never a fabricated number.
 - j. The payload is cached and logged to `ai_memory_log` (session + best-effort Snowflake table).
5. `render_ai_score_card()` or bespoke UI code renders the score, label, confidence, reasoning, risks, opportunities, and recommended actions.

3.4 Decision Workflow (Price Engine tab, full pipeline)



4 Technology Stack

Category	Technology	Evidence in Source
Language	Python 3 (Streamlit runtime)	Entire file is Python; import streamlit as st
UI Framework	Streamlit	st.set_page_config, st.tabs, st.columns, st.metric, st.dataframe, st.bar_chart, st.line_chart, st.scatter_chart, st.selectbox, st.slider, st.text_area, st.expander, st.radio
Compute / Data Platform	Snowflake, accessed via Snowpark	from snowflake.snowpark.context import get_active_session; session.sql(...)
Hosting Model	Streamlit-in-Snowflake (native app)	No external connection/auth code; session obtained natively
AI Platform	Snowflake Cortex AI	AI_COMPLETE, AI_CLASSIFY, AI_FILTER, AI_AGG, SNOWFLAKE.CORTEX.SUMMARIZE called via raw SQL
LLM Models (selectable)	llama3.1-70b (default), llama3.1-8b, mistral-large2, mistral-large, snowflake-arctic, reka-flash, llama3.3-70b, claude-3-5-sonnet	Cortex AI Diagnostics
Data Handling	pandas	st.selectbox options
Serialization	json, re (regex)	import pandas as pd; .to_pandas() on every Snowpark DataFrame
Hashing	hashlib (MD5)	parse_ai_json(), ai_memory_log(), _ai_cache_key() for cache-key generation (not for security)
Styling	Hand-written CSS injected via st.markdown(..., unsafe_allow_html=True), Google Fonts (Inter, JetBrains Mono)	Large <style> block at top of file
Charts	Streamlit-native charts only (st.bar_chart, st.line_chart, st.scatter_chart) plus one hand-rolled inline SVG gauge	No Plotly/Matplotlib import — consistent with the Streamlit-in-Snowflake sandbox, which historically has restricted third-party plotting libraries

Version information: No `requirements.txt`, `environment.yml`, or Snowflake app manifest was provided, so exact pinned versions of Streamlit, Snowpark, or pandas cannot be confirmed. This is noted as a documentation gap, not assumed.

5 Project Structure

Only a single Python file was supplied (referred to throughout this document as `streamlit_app.py`). There is no multi-folder repository structure to document. Internally, the file is organized into clearly delimited comment-banner sections, in this order:

#	Section (as delimited by comment banners in the file)	Approx. Role
1	Page Config	<code>st.set_page_config</code>
2	Global Styles	CSS theme (unmodified per an explicit comment)
3	Session State Initialization	<code>_init_state()</code> calls for ~14 keys
4	Snowflake Session	<code>get_active_session()</code>
5	Helpers (unchanged)	<code>section, divider, safe_pct_change, health_bar_html, get_capacity_modifier, get_capacity_recommendation</code>
6	Cortex AI Helpers	<code>ai_complete, ai_classify, ai_filter, ai_summarize, ai_agg_over_rows, parse_ai_json, ai_or_fallback</code>
7	AI Memory Layer	<code>ai_memory_log, get_ai_memory_context</code>
8	Central AI Reasoning Service	<code>ai_reason_score, ai_reason_score_batch, render_ai_score_card, plus payload coercion helpers</code>
9	PSI Helpers	<code>get_psi_margin, get_psi_recommendation, get_psi_score, get_psi_score_label, ai_psi_recommendation</code>
10	More AI recommendation helpers	<code>ai_capacity_recommendation, ai_win_probability_explanation, ai_discount_justification</code>
11	Hero Banner	Static HTML banner
12	Cortex AI Diagnostics	Model selector + connectivity test expander
13	Tabs 1-8	See Module Documentation chapter
14	Footer	Static HTML footer

There are no separate `pages/`, `utils/`, `models/`, or `tests/` directories in what was provided — this is a single monolithic script, which is called out explicitly in the Known Limitations chapter.

6 Module Documentation

Each Streamlit tab is treated as a module. Two internal helper functions defined at the top of the file (`get_psi_margin/get_capacity_modifier/etc.`) are documented once in the Function Documentation chapter and referenced here rather than repeated.

6.1 Module: Overview (Tab 1)

- **Purpose:** Landing dashboard with headline KPIs and lightweight analytics; explicitly commented as “pure KPIs/charts, no business rules” (i.e., no AI integration).
- **Inputs:** None from the user; reads directly from Snowflake.
- **Data sources:** `CUSTOMER_PRICING`, `CUSTOMER_AGREEMENTS`, `USER_SIMULATION_RESULTS`.
- **Outputs:** 4 KPI metrics (Products Tracked, Avg Cost, Avg Target Price, Avg Customer Price), a bar chart of target price by SKU, a bar chart of customer discounts, a profit table (`DISCOUNTED_PRICE - TOTAL_COST` and margin %), a “Most Profitable Products” ranking, and a table of the 20 most recent simulation runs.
- **Business logic:** $\text{PROFIT} = \text{DISCOUNTED_PRICE} - \text{TOTAL_COST}$; $\text{MARGIN \%} = \text{PROFIT} / \text{TOTAL_COST} \times 100$.
- **AI integration:** None (by design/comment).
- **Error handling:** Each block is wrapped in `try/except`, falling back to `st.info(...)` messages; an empty DataFrame is substituted so downstream code does not crash.

6.2 Module: Executive Decision Center (Tab 2)

- **Purpose:** “CEO / CFO level summary — consolidates every pricing module” with AI-generated insight layered on deterministic KPIs.
- **Data sources:** `CUSTOMER_PRICING`, `OPTIMIZED_PRICING_MATRIX`, `PRICE_SENSITIVITY`, `PLANT_CAPACITY`, `CUSTOMER_AGREEMENTS`, `DYNAMIC_SURCHARGES` (loaded but not otherwise referenced further in this file — **partially used**), all via a local `_safe_load()` wrapper that returns an empty DataFrame on any failure.
- **Sub-sections (12 total):**
 1. Executive KPI Cards — Total Revenue, Gross Profit, Avg Margin, Pricing Health (preview), Avg Win Prob., Active Customers. All deterministic aggregates except the “Pricing Health” preview, which is a simple mean of PSI/margin/win-probability inputs shown only as a placeholder ahead of the full AI score in section 10.
 2. Business Snapshot — customer/product/contract counts, average price/cost.
 3. Executive Alerts — candidate signals (margin leakage count, capacity $\geq 90\%$ count, win probability $< 40\%$ count, high-profit count $> 30\%$ margin) are computed deterministically, then gated through `AI_FILTER` (fail-open: if AI says “not significant” the alert is dropped; if AI is unavailable, the alert is still shown) and phrased via `AI_COMPLETE`.
 4. Top Opportunities — deterministic top-10-by-margin rows are sent in **one batched** `AI_COMPLETE` call that returns a JSON array of `{suggested_action, expected_benefit}` per row; a per-row deterministic fallback (`_fallback_action`) exists if the AI call fails.
 5. Top Risks — deterministic detection (margin leakage rows, negative-margin contracts, over-capacity plants, low-win-probability deals), each severity-labeled via `AI_CLASSIFY` into Critical/High/Medium/Low, with a hardcoded fallback label per risk type.
 6. Revenue Dashboard — bar charts by customer/product, and overall distribution (fully deterministic, no AI).
 7. Margin Dashboard — margin distribution, top/bottom 10 products by margin (deterministic).

8. Customer Insights — highest revenue/profit/discount/win-probability customer KPIs (deterministic), a profit-vs-revenue scatter, and an **AI Customer Segmentation** of the top 8 customers by revenue into one of {Strategic, Premium, Growth, Enterprise, Price Sensitive, At Risk} via one batched AI_COMPLETE call.
 9. Product Insights — best/worst/highest-revenue/highest-margin product KPIs (deterministic), plus **AI Product Classification** of the top 8 products by revenue into {Star, Core, Niche, Underperformer}.
 10. Pricing Health Score — fully AI-driven via the central `ai_reason_score(module="executive_health_score", decision_type="pricing_health_score")` pipeline; the model is explicitly instructed to weigh the input facts however it judges appropriate rather than using a fixed formula.
 11. Executive Recommendations — deterministic “signal” detection (underpriced SKUs, high-discount customers, over-capacity plants, low-margin combos, negative-margin contracts) converted into up to 5 AI-written recommendations with an AI-assigned severity (Critical/Warning/Positive); falls back to the raw deterministic signal strings if AI fails.
 12. Executive Summary — an AI-written 3–4 sentence narrative built strictly from the facts computed above (revenue, profit, top opportunity, highest risk, best product/customer, key recommendation), with the underlying facts always shown in an expander for traceability, and a deterministic markdown fallback if AI fails.
- **Limitations:** DYNAMIC_SURCHARGES is loaded into `surcharge_df` but never read again in this tab — **partially implemented**. The “Pricing Health (preview)” KPI in section 1 uses a different (simpler, unweighted-mean) calculation than the “real” AI-driven Pricing Health Score in section 10, and the two numbers can legitimately disagree; this is a **known inconsistency**, not a bug in the sense of a crash, but worth flagging to stakeholders.

6.3 Module: Price Engine (Tab 3, Python variable `tab3`)

- **Purpose:** The main working surface for finding a profitable selling price using PSI, capacity, win probability, and discount optimization.
- **Sub-modules, in on-page order:**
 1. **Product/demand selection** — SKU dropdown (from `CALCULATED_STANDARD_COSTS`) and a monthly demand number input.
 2. **Customer Price Lookup** — joins `CUSTOMER_PRICING` and `OPTIMIZED_PRICING_MATRIX` for a chosen SKU + customer, returning cost, floor/target/ceiling price, and discounted price; persisted in `st.session_state["cp_result"]`.
 3. **Price Sensitivity Analysis (PSI)** — reads `PRICE_SENSITIVITY` for the customer chosen above; margin % and full risk/opportunity payload are produced by `ai_reason_score(module="price_sensitivity", decision_type="psi_margin_strategy", numeric_fields={"margin_pct": ...})`. The **only** deterministic step is `optimized_price = cost × (1 + margin_pct/100)`, and (if a customer lookup was already run) reapplying that customer’s back-calculated discount %. On AI failure, a clearly labeled **10 % default margin** is used and a warning is shown (not a silent guess).
 4. **Plant Capacity Analysis (CUM)** — reads `PLANT_CAPACITY`; the price modifier (range instructed as –20 to +20 %) and a capacity health score/label come from `ai_reason_score(module="capacity", decision_type="capacity_modifier", ...)`. Deterministic step: `capacity_adjusted_price = optimized_price × (1 + modifier_pct/100)`, then the same customer discount is re-applied.
 5. **Win Probability Predictor** — reads `COMPETITOR_PRICING` (average price for the SKU); the headline win probability (0–100) is the score field of `ai_reason_score(module="win_prob", decision_type="deal_win_probability")`. A **separate, intentionally lightweight deterministic formula** (`compute_win_probability`) is retained *only* to drive the interactive ± 20 % “Price Sensitivity Simulation” chart, because re-calling Cortex for

every point of every chart redraw would be slow; this is explicitly commented in the source as not being used for “the” headline number.

6. **Smart Discount Optimizer** — evaluates 5 fixed discount levels (0/2/5/8/10 %). Win probability per level comes from **one batched** `ai_reason_score_batch(module="discount_optimizer", decision_type="win_probability_per_discount")` call (not 5 separate calls). Revenue/profit per level are deterministic multiplications once win probability is known. The “best” scenario is *also* an AI judgment (`decision_type="scenario_selection"`) that reasons over the whole table rather than a fixed “if win $\geq$ 80 % maximize profit” rule; if AI is unavailable, the fallback is the scenario with the highest win probability (a neutral tiebreak, chosen specifically to avoid reintroducing a hardcoded business rule). `ai_discount_justification()` supplies the final one-line business justification text.
 7. **Run Price Optimization** (button, bottom of tab) — a **fully deterministic**, independent calculation: for margin % in {20,25,...,50}, $\text{target_price} = \text{cost} \times (1 + \text{margin}/100)$, $\text{demand_factor} = 1 - (\text{margin} - 20) \times 0.02$, $\text{expected_demand} = \max(\text{int}(\text{base_demand} \times \text{demand_factor}), 1)$, $\text{revenue} = \text{target_price} \times \text{expected_demand}$, $\text{profit} = (\text{target_price} - \text{cost}) \times \text{expected_demand}$; the margin with the highest profit is recommended. Results are appended to `PRICING_ENGINE_DB.CORE_OUTPUT.OPTIMIZATION_RESULTS`. **This sub-module does not use AI at all** and is architecturally independent of the PSI/CUM/Win-Probability chain above it on the same tab.
- **Note on internal consistency:** The Price Engine tab contains *two different, independently-run pricing models* — the AI-driven PSI→CUM→WinProb→Discount chain, and the classic deterministic elasticity-sweep optimizer — that are not reconciled with each other. Both are fully functional but a user could see two different “recommended price” figures for the same SKU depending on which sub-module they use. This is documented as a **known limitation**, not an error.

6.4 Module: Simulation Hub (Tab 4)

- **Purpose:** Ad-hoc, fully deterministic “what-if” pricing scenarios; explicitly no AI in this tab.
- **Custom scenario:** $\text{adjusted_cost} = \text{cost} \times (1 + \text{steel}\%/100 + \text{fuel}\%/100)$; $\text{target_price} = \text{adjusted_cost} \times (1 + \text{target_margin}\%/100)$; $\text{customer_price} = \text{target_price} \times (1 - \text{discount}\%/100)$; $\text{profit} = \text{customer_price} - \text{cost}$. Saved to `USER_SIMULATION_RESULTS`.
- **Pre-defined scenario comparison:** looks up a `SCENARIO_ID` against `PRICING_ENGINE_DB.CORE_INTER` (for the picklist) and `PRICING_ENGINE_DB.CORE_OUTPUT.SIMULATED_PRICING` (for the values), and renders a bar-chart comparison across all stored scenarios.
- **Simulation history:** raw table dump of `USER_SIMULATION_RESULTS` ordered by `CREATED_AT DESC`.

6.5 Module: Digital Twin (Tab 5)

- **Purpose:** Model a future market shock (steel/fuel cost change, demand change) against a fixed assumed baseline margin.
- **Deterministic math:** `BASE_MARGIN = 1.35` (i.e., a hardcoded 35 % baseline margin assumption — **not** derived from PSI); $\text{base_price} = \text{cost} \times 1.35$; $\text{new_cost} = \text{cost} \times (1 + \text{steel}\%) \times (1 + \text{fuel}\%)$; $\text{new_price} = \text{new_cost} \times 1.35$; $\text{new_demand} = \max(\text{int}(\text{base_demand} \times (1 + \text{demand_change}\%)), 1)$; revenue/profit computed from $\text{price} \times \text{demand}$ for both states. Saved to `DIGITAL_TWIN_RESULTS`.
- **AI integration — Margin Leakage:** $\text{future_margin} = (\text{new_price} - \text{new_cost}) / \text{new_price} \times 100$ is computed deterministically, then passed as context to `ai_reason_score(module="digital_twin")`.

decision_type="margin_leakage"), which returns a health score/label, reasoning, risks, opportunities, and recommended actions, rendered via render_ai_score_card.

- **Limitation:** the 35 % baseline margin is a fixed constant independent of the PSI-driven margin logic used elsewhere in the app — another example of two pricing models coexisting without reconciliation.

6.6 Module: AI Advisor (Tab 6)

- **Purpose:** Reads the **single most recent** row from DIGITAL_TWIN_RESULTS (ORDER BY CREATED_AT DESC LIMIT 1) and produces an AI-driven strategic read-out of that scenario. If no Digital Twin simulation has ever been run, the tab shows a warning and stops.
- **Deterministic inputs:** profit_change = safe_pct_change(new_profit, base_profit); revenue_change = safe_pct_change(new_revenue, base_revenue).
- **AI integration:** ai_reason_score(module="advisor", decision_type="scenario_health") produces the health score/label (replacing what an in-code comment says was previously a fixed "+25/+10/-15/-25 point" rule). Recommended actions are generated by a dedicated AI_COMPLETE call requesting a JSON array of up to 4 short recommendations, with a deterministic if/elif fallback list if AI fails. The Executive Summary text is produced by SNOWFLAKE.CORTEX.SUMMARIZE via ai_summarize(), again with a deterministic fallback template.

6.7 Module: Contract Analyzer (Tab 7)

- **Purpose:** Assess how a customer/SKU contract's margin would erode under a steel/fuel cost shock.
- **Deterministic math:** current_margin = (selling_price - cost)/selling_price × 100; future_cost = cost × (1+steel%) × (1+fuel%); future_margin = (selling_price - future_cost)/selling_price × 100; margin_loss = current_margin - future_margin.
- **AI integration:** ai_reason_score(module="contract", decision_type="contract_risk") returns the risk score/label and reasoning (replacing what a comment states was previously a fixed future_margin × 4 formula with hardcoded 10 %/20 % thresholds). Recommended actions are a dedicated AI_COMPLETE JSON-array call, with an if/elif fallback keyed on future_margin thresholds (10 %, 20 %).

6.8 Module: Competitor Pricing (Tab 8)

- **Purpose:** Benchmark a SKU's target price against four user-entered named competitors, run a "price war" simulator, review a (synthetic) 6-month trend, and produce an executive summary.
- **Inputs:** SKU + monthly volume, plus 4 competitor name/price pairs entered directly in the UI (there is no competitor database table read here — all four competitor prices are manually typed by the user each session).
- **Deterministic helpers (also used for real-time UI):** safe_pct(a,b), get_position_fallback(gap) (fixed >10/>3/>-3 thresholds → Expensive/Premium/Competitive/Discount), health_score(gap) = clamp(100 - |gap×5|, 0, 100).
- **AI-driven equivalents:** get_position(gap) calls AI_CLASSIFY to choose the same four labels (falls back to get_position_fallback on failure); the main "Analyze Market Position" button computes the headline competitive health score via ai_reason_score(module="competitor", decision_type="market_position") instead of the health_score() formula.
- **Deliberate dual-system design:** the deterministic health_score()/get_position_fallback() functions are explicitly retained (per source comments) for the **Price War Simulator**

slider and the **Market Position Matrix** scatter plot, both of which redraw on every UI interaction — a live Cortex call per drag-frame would be impractical, so those two visualizations intentionally stay on the fast deterministic path while the main analysis button uses the full AI pipeline.

- **Historical Trends sub-tab — important limitation:** the “6-Month Competitor Price Trends” chart is **not** built from real historical data. It is generated by multiplying each current price by a fixed hardcoded array of ratios (e.g. [.92, .95, .97, .99, 1.01, 1]), i.e. a **synthetic backward interpolation**, not an actual time series read from any table. This must be clearly disclosed to any business user viewing that chart.
- **AI integration on this sub-tab:** AI_AGG is used (via ai_agg_over_rows) to summarize the (synthetic) trend across all five price series in one Cortex call.
- **Executive Summary sub-tab:** combines the deterministic KPI block with an SNOWFLAKE.CORTEX.SUMMARIZE-generated narrative and one final AI_COMPLETE “raise/hold/lower” recommendation sentence, both with deterministic fallbacks.

7 Detailed Function Documentation

This chapter documents every top-level function defined in the file. Functions are grouped by the section they appear in.

7.1 UI / Formatting Helpers

Function	Purpose	Parameters	Returns	Notes
<code>section(icon, title)</code>	Renders a styled section header (icon chip + heading)	<code>icon: str</code> , <code>title: str</code>	None (writes HTML via <code>st.markdown</code>)	Used throughout every tab
<code>divider()</code>	Renders a styled <code><hr></code>	none	None	Used throughout every tab
<code>safe_pct_change(base_val)</code>	Percent change guarding against divide-by-zero	<code>new_val: float</code> , <code>base_val: float</code>	float (0.0 if <code>base_val==0</code>)	Used in Executive Alerts context, AI Advisor, Win-Probability market-gap calc
<code>health_bar_html(score, color, status)</code>	Renders an HTML/CSS progress-bar widget	<code>score: int/float (0-100)</code> , <code>color: str</code> , <code>status: str</code>	str (HTML)	Reused by nearly every scored module for visual consistency

7.2 Legacy Deterministic Rule Functions (superseded by AI, retained in source)

Function	Purpose	Status
<code>get_capacity_modifier(utilized)</code>	Fixed lookup: <code><50 %→-5 %</code> , <code><70 %→0 %</code> , <code>≤85 %→+5 %</code> , else <code>+10 %</code>	Defined but not called anywhere in the rendered tabs — the live Capacity Analysis section uses <code>ai_reason_score</code> exclusively. Partially implemented / dead code.
<code>get_capacity_recommendation(utilized)</code>	Fixed text with the same thresholds	Only ever referenced as the documented “fallback text” inside the docstring of <code>ai_capacity_recommendation</code> , but <code>ai_capacity_recommendation</code> itself is also never called from any tab. Dead code.
<code>ai_capacity_recommendation(utilized)</code>	Would generate an AI_COMPLETE capacity recommendation	Defined but never invoked — the live tab uses <code>ai_reason_score</code> directly instead. Dead code / superseded.

Function	Purpose	Status
<code>get_psi_margin(psi)</code>	Fixed PSI→margin lookup table ($\geq 0.80 \rightarrow 8\%$, ... $< 0.20 \rightarrow 40\%$)	Defined but not called — the live PSI section uses <code>ai_reason_score</code> . Still shown to the user as a static “Margin Tier Reference” table for context. Partially implemented / reference-only.
<code>get_psi_recommendation(psi)</code>	Fixed PSI recommendation text	Only referenced as documented fallback text inside <code>ai_psi_recommendation</code> ’s docstring; <code>ai_psi_recommendation</code> itself is never called. Dead code.
<code>get_psi_score(psi) /</code> <code>get_psi_score_label(score)</code> <code>ai_psi_recommendation(...)</code>	Fixed $100 - \text{psi} \times 100$ score + label lookup Would generate an AI_COMPLETE PSI recommendation	Defined but not called. Dead code. Defined but never invoked in the current PSI section (which calls <code>ai_reason_score</code> directly). Dead code / superseded.
<code>ai_win_probability_explanation(...)</code>	Would generate an AI_COMPLETE win-probability explanation	Defined but never invoked — the live Win Probability section uses <code>ai_reason_score</code> directly and reads reasoning off its payload. Dead code / superseded.
<code>ai_discount_justification(...)</code>	Generates the Smart Discount Optimizer’s justification text	Actively called in the Smart Discount Optimizer section.

Architectural observation: the presence of this many superseded-but-undeleted helper functions is strong evidence the codebase went through an in-place “v1 → v2” migration from fixed business rules to AI-driven scoring, without a cleanup pass removing the v1 functions. This is flagged as **Technical Debt** in Document 2.

7.3 Cortex AI Low-Level Wrappers

Function	Purpose	Parameters	Returns	Failure Behavior
<code>ai_complete(prompt, model=None, cache_key=None)</code>	Free-text generation via AI_COMPLETE	<code>prompt: str</code> , optional model override, optional cache key	<code>str \ None</code>	Returns None on any exception; records error in <code>st.session_state["ai_last_"]</code> and increments a fail counter

Function	Purpose	Parameters	Returns	Failure Behavior
<code>ai_classify(text, categories, cache_key=None)</code>	Single-label classification via AI_CLASSIFY	<code>text: str</code> , <code>categories: list[str]</code>	<code>str</code> \ <code>None</code> (chosen label)	Same as above
<code>ai_filter(text, cache_key=None)</code>	Boolean significance gate via AI_FILTER	<code>text: str</code>	<code>bool</code> \ <code>None</code>	Same as above; callers are told to fail-open (treat <code>None</code> as “show it”) for alerts
<code>ai_summarize(text, cache_key=None)</code>	Narrative condensation via SNOWFLAKE.CORTEX.SUMMARIZE	<code>text: str</code>	<code>str</code> \ <code>None</code>	Same as above
<code>ai_agg_over_rows(question, cache_key=None)</code>	Aggregates, reasoning across up to 50 rows in one Cortex call via AI_AGG over a constructed VALUES table	<code>row_texts: list[str]</code> , <code>question: str</code>	<code>str</code> \ <code>None</code>	Same as above; returns <code>None</code> immediately if <code>row_texts</code> is empty
<code>_ai_cache_key(*parts)</code>	MD5 hash of concatenated parts, used purely as a session cache key (not for security)	any	<code>str</code>	n/a
<code>_sql_escape(text)</code>	Escapes backslashes and single quotes before string interpolation into SQL	<code>text</code>	<code>str</code>	See Security chapter for limitations of this approach
<code>parse_ai_json(raw_text)</code>	Multi-strategy JSON extraction from LLM output (direct parse → strip code fences → regex-extract {...}/[...] → strip trailing commas)	<code>raw_text: str</code>	parsed object or <code>None</code>	Returns <code>None</code> only if every strategy fails
<code>ai_or_fallback(ai_text, fallback)</code>	One-liner, convenience: use AI text if truthy, else fallback	any, any	either input	n/a
<code>_record_ai_error</code> <code>/_record_ai_ok</code>	Update <code>st.session_state["ai_last_error"]</code> , <code>["ai_last_raw"]</code> , <code>["ai_call_stats"]</code>	—	<code>None</code>	Powers the Cortex AI Diagnostics panel

7.4 AI Memory Layer

Function	Purpose	Returns
<code>ai_memory_log(module, decision_type, context_summary, result)</code>	Appends a decision to <code>st.session_state["ai_memory"]</code> (capped at last 200 entries) and best-effort INSERTs into <code>PRIC-ING_ENGINE_DB.CORE_OUTPUT.AI_MEMORY_LOG</code> ; silently ignores the INSERT failure if the table is missing/inaccessible	None
<code>get_ai_memory_context(module, decision_type=None, limit=3)</code>	Returns up to <code>limit</code> prior decisions for the module (in-session first, falling back to a Snowflake SELECT if none found in-session) as a bullet-point string for prompt context	str

7.5 Central AI Reasoning Service

Function	Purpose	Key Parameters	Returns
<code>_coerce_score_payload(payload)</code>	Limitation: only rejects a payload if score cannot be parsed as a number (0-100, tolerant of "85"/"85%" strings); all other fields default gracefully	<code>payload: dict</code>	normalized dict or None
<code>_neutral_score_payload(payload)</code>	Explicit "Unavailable" placeholder — never a disguised hardcoded business score	<code>reason: str</code>	dict with <code>ai_generated: False</code>
<code>ai_reason_score(module, decision_type, context_facts, extra_instruction="", numeric_fields=None, cache_key=None)</code>	The core reusable pipeline: build context → retrieve AI memory → prompt for strict JSON → parse → coerce → one repair retry on failure → cache → log to memory	see above	dict with at minimum score, label, confidence, reasoning, risks, opportunities, recommended_actions, ai_generated, plus any requested numeric_fields

Function	Purpose	Key Parameters	Returns
<code>ai_reason_score_batch</code> <code>decision_type,</code> <code>items_context, extra_instruction="",</code> <code>cache_key=None)</code>	Same idea as batched across N items in one Cortex call (used for discount-level win-probability sweeps)	<code>items_context: list[str]</code>	<code>list[dict]</code> , one per input item, each independently defaulted to a neutral payload if the AI response can't be matched to that index
<code>render_ai_score_card</code> <code>title_prefix="AI Score")</code>	<code>\$payload</code> render: health bar + reasoning + 3-column Risks/Opportunities/Recommended-Actions layout; on <code>ai_generated=False</code> shows a warning plus a "Cortex AI Diagnostics" expander with the last raw error/response and common-cause guidance	<code>payload: dict</code>	None (renders to Streamlit)

7.6 PSI / Capacity / Simulation Math

Function	Purpose	Formula
<code>get_psi_margin(psi)</code>	Legacy margin tier lookup (see above — not called live)	See threshold table in Business Logic chapter
<code>get_psi_score(psi)</code>	Legacy score (not called live)	$\text{clamp}(100 - \text{psi} \times 100, 0, 100)$
<code>compute_win_probability(psi, discount_pct, capacity_mod_pct, competitor_avg)</code>	Actively used lightweight estimator for the interactive price-sweep chart only	$\text{base}(50) + (0.5 - \text{psi}) \times 20 + \text{competitor_impact}(\pm 20, \text{from gap\%}) + \text{discount_impact}(0-15) + \text{capacity_impact}(-0.5 \times \text{capacity_mod_pct})$, clamped 0-100
<code>win_probability_gauge_html</code> <code>color_hex, status_label)</code>	Renderers a hand-built semi-circle SVG gauge (no chart library)	Pure SVG path math (<code>math.cos/math.sin</code>)

7.7 Competitor Tab Local Helpers (defined inside the `with tab8: block`)

Function	Purpose
<code>safe_pct(a, b)</code>	Local re-implementation of a percent-change helper, functionally similar to the module-level <code>safe_pct_change</code> but with a/b argument order swapped — see Technical Debt
<code>get_position_fallback(gap)</code>	Deterministic 4-tier position label (see Module doc above)
<code>get_position(gap)</code>	AI_CLASSIFY-driven version of the same, with fallback to <code>get_position_fallback</code>
<code>health_score(gap)</code>	Deterministic competitive health score ($100 - \text{gap} \times 5 $, clamped)
<code>health_bar(score, color, label)</code>	Local HTML progress bar (distinct implementation from the module-level <code>health_bar_html</code> , functionally similar but a separate code path — see Technical Debt)
<code>alert_chip(text) / opp_chip(text)</code>	Small styled HTML chips for alerts/opportunities lists

8 Database Documentation

No DDL was supplied. Every table below is **[Inferred from usage]** — columns are exactly the ones referenced in a SELECT, WHERE, INSERT, or DataFrame column-access in the source. Where a query uses SELECT *, only columns that are subsequently accessed by name in Python are listed with confidence; any other columns in the real table are unknown and not claimed here.

Table	Schema (Inferred)	Referenced Columns	How the App Uses It
CORE_OUTPUT.CUSTOMER_PRICING	per-customer, per-SKU priced output	SKU, CUSTOMER_ID, PRODUCT_FAMILY, TOTAL_COST, TARGET_PRICE, DISCOUNTED_PRICE, FLOOR_PRICE, CEILING_PRICE	Read in Overview, Executive Decision Center, Customer Price Lookup (joined to OPTIMIZED_PRICING_MATRIX), Contract Analyzer
CORE_INPUT.CUSTOMER_AGREEMENTS	Agreements customer terms	CUSTOMER_ID, DISCOUNT_PCT (assumed name DISCOUNT_PCT), CONTRACT_ID, and (in Tab 2 only) a MARGIN_PCT column used to detect negative-margin contracts	Read in Overview (discount chart), Executive Decision Center (contract counts, risk detection), Customer Price Lookup picklist
CORE_OUTPUT.OPTIMIZED_PRICING_MATRIX	Product-level pricing bands + predicted win probability	SKU, PRODUCT_FAMILY, TOTAL_COST, FLOOR_PRICE, TARGET_PRICE, CEILING_PRICE, WIN_PROBABILITY (assumed name), CUSTOMER_ID (only in the Competitor tab's own separate query)	Read in Customer Price Lookup join, Executive Decision Center KPI/alerts, Competitor Pricing (target price source)
CORE_OUTPUT.PRICE_SENSITIVITY	per-customer price sensitivity index	CUSTOMER_ID, CUSTOMER_SEGMENT, PRICE_SENSITIVITY_INDEX (Price Engine tab); a differently-named PSI_SCORE column is assumed in Tab 2's KPI code — naming inconsistency , see Technical Debt	Read in Price Engine's PSI section and Executive Decision Center KPIs/health score

Table	Schema (Inferred)	Referenced Columns	How the App Uses It
CORE_INPUT.PLANT_CAPACITY	Plant level utilization	PLANT_ID, PLANT_NAME, CURRENT_UTILIZATION, MAX_CAPACITY, AVAILABLE_CAPACITY; Tab 2 additionally assumes a UTILIZATION_PCT column — possible naming inconsistency with CUR-RENT_UTILIZATION used in Tab 3 , see Technical Debt (<i>unknown</i>)	Read in Price Engine's Capacity Analysis section and Executive Decision Center alerts/risks
CORE_OUTPUT.DYNAMIC_SURCHARGES	SURCHARGES loaded but never referenced again		Loaded in Tab 2 (surcharge_df) but not used anywhere downstream in the provided file
CORE_INPUT.CALCULATED_STANDARD_COST	DESIGN and standard cost	SKU, TOTAL_COST	Read in Price Engine, Simulation Hub, Digital Twin (SKU pickers and cost lookups)
CORE_INTERNAL.SIMULATION_SCENARIO	Non-Simulation scenario catalogue	SCENARIO_ID, SCENARIO_NAME	Read in Simulation Hub for the scenario picklist
CORE_OUTPUT.SIMULATED_PRICING	Pre-priced scenario results	SKU, SCENARIO_ID, SCENARIO_NAME, TOTAL_COST, TARGET_MARGIN_PCT, CUSTOMER_DISCOUNT_PCT, SIMULATED_TARGET_PRICE, SIMULATED_CUSTOMER_PRICE	Read in Simulation Hub (scenario detail + all-scenario comparison chart)
CORE_OUTPUT.USER_SIMULATION_RESULTS	Log of all user-run simulations	SCENARIO_NAME, SKU, TOTAL_COST(cost), TARGET_MARGIN_PCT, CUSTOMER_DISCOUNT_PCT, STEEL_SURCHARGE, FUEL_SURCHARGE, TARGET_PRICE, CUSTOMER_PRICE, CREATED_AT	Written by Simulation Hub's custom-scenario button; read by Overview and Simulation Hub history views

Table	Schema (Inferred)	Referenced Columns	How the App Uses It
CORE_OUTPUT.OPTIMIZATION_RESULTS	“Price Optimization” runs	SKU, COST, MARGIN_%, TARGET_PRICE, EXPECTED_DEMAND, REVENUE, PROFIT, CREATED_AT	Written (append) by the Price Engine’s elasticity-sweep optimizer; not read back anywhere in this file
CORE_OUTPUT.DIGITAL_TWIN_RESULTS	Digital Twin runs	SCENARIO_NAME, SKU, COST, NEW_COST, BASE_PRICE, NEW_PRICE, BASE_DEMAND, NEW_DEMAND, BASE_REVENUE, NEW_REVENUE, BASE_PROFIT, NEW_PROFIT, CREATED_AT	Written by Digital Twin tab; read (latest row only) by the AI Advisor tab
CORE_INPUT.COMPETITOR_PRICING	Competitor average price per SKU	SKU, COMPETITOR_AVG_PRICE	Read (aggregated with AVG()) by the Win Probability Predictor only — note: the Competitor Pricing tab itself does not read this table; it uses manually-typed prices instead
CORE_OUTPUT.AI_MEMORY_LOG	AI decision journal, created by the app itself	MODULE, DECISION_TYPE, CONTEXT_SUMMARY, RESULT_SUMMARY, CREATED_AT	CREATE TABLE IF NOT EXISTS is issued once per session (best-effort, swallowed on privilege failure); written by ai_memory_log, read by get_ai_memory_context

No primary/foreign key constraints, indexes, or relationship diagrams are present in the source, because DDL was not supplied. Any ER diagram would have to be produced from the actual Snowflake schema, not from this application file, and is therefore intentionally omitted rather than fabricated.

9 SQL Documentation

All SQL is issued as literal, Python-f-string-interpolated `session.sql(...)` calls (no stored procedures, no parameterized bind variables, no dbt models). Representative categories:

Query Type	Example (paraphrased)	Purpose	Optimization Notes
Full-table pulls	<code>SELECT * FROM CORE_OUTPUT.CUSTOMER_PRICING</code>	Populate Overview	No column pruning or filtering — pulls the entire table into pandas every rerun; for large tables this is a performance concern (see Performance chapter)
Filtered single-row lookup	<code>SELECT ... FROM CUSTOMER_PRICING c JOIN OPTIMIZED_PRICING_MATRIX p ... WHERE p.SKU='{sku}' AND c.CUSTOMER_ID='{cust}' LIMIT 1</code>	Customer Price Lookup	Values are selectbox-constrained (drawn from the same tables), which limits — but does not formally eliminate — injection risk; see Security chapter
Aggregate query	<code>SELECT AVG(COMPETITOR_AVG_PRICE) FROM COMPETITOR_PRICING WHERE SKU='{sku}'</code>	Win Probability Market-gap calculation	Single aggregate, cheap
Cortex scalar function calls	<code>SELECT AI_COMPLETE('{model}', '{prompt}') AS RESP</code>	Every AI narrative/score	Every call is memoized in <code>st.session_state["ai_cache"]</code> so an identical prompt is never re-sent within a session
Cortex classify	<code>SELECT AI_CLASSIFY('{text}', [{categories}]):labels[0] AS LABEL</code>	Risk severity, market position, customer/product segmentation	—
Cortex filter	<code>SELECT AI_FILTER('{text}') AS FLAG</code>	Alert significance gating	—
Cortex aggregate	<code>SELECT AI_AGG(COL, '{question}') FROM (SELECT COLUMN1 AS COL FROM VALUES (...))</code>	Competitor trend summary across up to 50 constructed rows	Values are capped at 50 rows in <code>ai_agg_over_rows</code> to bound the constructed VALUES clause size
Cortex summarize	<code>SELECT SNOWFLAKE.CORTEX.SUMMARIZE('{text}') AS SUMMARY</code>	AI Advisor & Executive narratives	—

Query Type	Example (paraphrased)	Purpose	Optimization Notes
Inserts/writes	INSERT INTO USER_SIMULATION_RESULTS VALUES (...) and session.create_dataframe(df).write.mode("append").save_as_table(...)	Persisting simulation/optimization runs	Two different write patterns are used for conceptually similar "save a run"
DDL	CREATE TABLE IF NOT EXISTS CORE_OUTPUT.AI_MEMORY_LOG (...)	Bootstraps the AI memory table on first use	Technical Debt Wrapped in try/except; a missing-privilege failure is swallowed silently (recorded to diagnostics, not surfaced to the end user directly)

10 AI Documentation

10.1 AI_COMPLETE

The workhorse function for every free-text generation task: alert phrasing, top-opportunity actions, executive recommendations/summaries, PSI/capacity/win-probability/discount narratives (legacy path), advisor recommendations, contract recommendations, competitor recommendations, and the JSON payloads consumed by the central reasoning pipeline. Default model is llama3.1-70b, overridable at runtime via the Cortex AI Diagnostics panel (which also clears the AI cache when the model changes, so stale cached answers from a different model are never served).

10.2 AI_CLASSIFY

Used for: risk severity classification (Critical/High/Medium/Low) in Executive Alerts/Top Risks, customer segmentation (legacy ai_reason_score path additionally supersedes this for the main health scores), and market-position labeling (Expensive/Premium/Competitive/Discount) in Competitor Pricing.

10.3 AI_FILTER

Used exclusively as a **significance gate**: deciding whether a detected numeric signal (margin leakage count, capacity overload, competitive pricing gap) is worth surfacing as an executive alert. Both call sites are explicitly documented in source comments as **fail-open** — if AI_FILTER is unavailable, the alert is shown rather than hidden, on the reasoning that hiding a potentially important signal is worse than an extra false positive.

10.4 AI_AGG

Used once: to summarize a competitive pricing trend across up to 50 rows constructed from a VALUES clause, in a single round trip instead of one call per row.

10.5 SNOWFLAKE.CORTEX.SUMMARIZE

Used twice: the AI Advisor's Executive Summary, and the Competitor Pricing tab's Executive Summary narrative.

10.6 AI Memory

A lightweight decision journal. Every call to the central ai_reason_score pipeline logs its module, decision type, a 500-character context summary, and an 800-character result summary to st.session_state["ai_memory"] (capped at 200 entries) and best-effort to the AI_MEMORY_LOG Snowflake table. When a new decision of the same module/decision-type is being made, up to 3 prior entries are retrieved and injected into the prompt as "prior decisions (for consistency, not binding)" — i.e., it is a soft consistency aid, not a hard constraint on the model's output.

10.7 Caching

st.session_state["ai_cache"] is a flat dictionary keyed by an MD5 hash of the call's inputs (module/decision type/facts/prompt as applicable). Every one of ai_complete, ai_classify, ai_filter, ai_summarize, ai_agg_over_rows, and ai_reason_score/ai_reason_score_batch

checks this cache before making a network call. The cache is **session-scoped only** — it is not persisted across browser sessions or Streamlit reruns of a fresh session, and it is fully cleared whenever the Cortex model override is changed.

10.8 Reasoning Engine / Decision Engine

`ai_reason_score` is the single reusable “Build Context → Retrieve Historical Context → AI_REASON/AI_SCORE → AI_EXPLAIN → AI_VALIDATE (retry once)” pipeline, called by name from 9 distinct modules: `executive_health`, `price_sensitivity`, `capacity`, `win_probability`, `discount_optimizer` (both the per-level batch and the scenario-selection call), `digital_twin`, `advisor`, `contract`, and `competitor`. `ai_reason_score_batch` is the batched sibling, used once, for per-discount-level win probability.

10.9 Where Every AI Function Is Used (Cross-Reference)

Module (Tab)	AI Functions Invoked
Overview	None
Executive Decision Center	AI_FILTER, AI_COMPLETE (alerts); AI_COMPLETE JSON batch (top opportunities); AI_CLASSIFY (top risks); AI_COMPLETE JSON batch (customer segmentation, product classification); <code>ai_reason_score</code> →AI_COMPLETE (health score); AI_COMPLETE JSON (recommendations); AI_COMPLETE (executive summary narrative)
Price Engine	<code>ai_reason_score</code> (PSI margin, capacity modifier, win probability, discount-level batch, scenario selection); <code>ai_discount_justification</code> →AI_COMPLETE
Simulation Hub	None
Digital Twin	<code>ai_reason_score</code> (margin leakage)
AI Advisor	<code>ai_reason_score</code> (scenario health); AI_COMPLETE JSON (recommended actions); SNOWFLAKE.CORTEX.SUMMARIZE (executive summary)
Contract Analyzer	<code>ai_reason_score</code> (contract risk); AI_COMPLETE JSON (recommended actions)
Competitor Pricing	<code>ai_reason_score</code> (market position); AI_CLASSIFY (position label in the price-breakdown table); AI_FILTER + AI_COMPLETE (alerts/opportunities); AI_COMPLETE JSON (recommendations); AI_AGG (trend insight); SNOWFLAKE.CORTEX.SUMMARIZE + AI_COMPLETE (executive summary + final recommendation)

11 User Interface Documentation

11.1 Global Chrome

- Wide layout, sidebar collapsed by default, custom dark “glassmorphism” theme injected via a single large `<style>` block (Inter/JetBrains Mono fonts from Google Fonts, CSS variables for a cyan/blue/violet/pink palette).
- A hero banner (“Pricing Intelligence Platform”) with 4 static feature pills (Cost-to-Price Engine, Demand Elasticity, Customer Agreements, AI Strategy Advisor).
- A collapsible **Cortex AI Diagnostics** panel above the tabs: a model-selector dropdown (8 models), a “Test Cortex Connection” button that runs a literal `AI_COMPLETE(' <model>', 'Reply with exactly the word: OK')` probe and reports success/empty/error, a running count of successful/failed Cortex calls for the session, and the last captured error string.
- 8 top-level tabs (icons + labels as coded): Overview, Executive Decision Center, Price Engine, Simulation Hub, Digital Twin, AI Advisor, Contract Analyzer, Competitor Pricing.
- A static footer crediting “Snowflake Cortex AI · Streamlit-in-Snowflake”.

11.2 Per-Tab Widget Inventory

Tab	Selects/Inputs	Buttons	Charts	Tables
Overview	—	—	1 bar (target price), 1 bar (discounts)	Profit table, top-profit table, simulation history table
Executive Decision Center	—	—	2 bar (revenue by customer/product), 1 bar (distribution), 2 bar (margin top/bottom 10), 1 scatter (profit vs revenue)	Top opportunities, top risks, customer segmentation, product classification
Price Engine	SKU, demand, SKU+customer (lookup), plant	Lookup, Run Price Optimization	Price-vs-win-prob/revenue/profit line charts (sweep), discount-level bar/line charts	Customer lookup result, margin-tier reference, capacity-modifier reference, discount simulation matrix, full optimization results
Simulation Hub	Scenario name, SKU, 4 sliders (margin/discount/steel/fuel), scenario picklist	Run Simulation, Run Pre-Defined Scenario	2 bar (scenario comparison)	Scenario detail, simulation history

Tab	Selects/Inputs	Buttons	Charts	Tables
Digital Twin	SKU, scenario name, demand, 3 sliders (steel/fuel/demand change)	Run Digital Twin	—	Current-vs-future comparison table
AI Advisor	—	—	—	— (health bar + text areas only)
Contract Analyzer	Customer, SKU, 2 sliders (steel/fuel change)	Analyze Contract	—	— (health bar + text area)
Competitor Pricing	SKU, monthly volume, 4× competitor name/price, radio (Overview/Trends/Price War/Exec Summary), price-war slider	Analyze Market Position	Bar (price distribution), scatter (position matrix), 3 line charts (trend tab), bar/line (price-war sim)	Market price breakdown, trend summary, simulated prices, executive summary

11.3 Session-State Persistence Pattern

A deliberate UI-engineering pattern is used throughout: because Streamlit reruns the entire script on every widget interaction, every “Run X” button’s result is written into a dedicated `st.session_state` key immediately after computation, and the **rendering code is a separate block that reads from session state**, not from the button’s local variables. This means a slider tweak elsewhere on the page does not wipe a previously computed result — a correctness fix that is explicitly called out in the source’s top-of-file comment block.

12 Business Logic Documentation

12.1 Pricing Formulas (Deterministic — never AI-generated)

Formula	Where Used
PROFIT = DISCOUNTED_PRICE - TOTAL_COST	Overview, Executive Decision Center
MARGIN % = PROFIT / TOTAL_COST × 100	Overview, Executive Decision Center
Optimized Price = Cost × (1 + AI-judged margin %)	Price Engine — PSI section
Capacity-Adjusted Price = Optimized Price × (1 + AI-judged modifier %)	Price Engine — Capacity section
Final Price = Capacity-Adjusted Price × (1 - customer discount %)	Price Engine (chained through PSI→CUM)
Expected Revenue = Final Price × Demand × Win-Probability%	Price Engine — Win Probability
Expected Profit = (Final Price - Cost) × Demand × Win-Probability%	Price Engine — Win Probability
Adjusted Cost = Cost × (1 + Steel% + Fuel%)	Simulation Hub
Target Price = Adjusted Cost × (1 + Target Margin%); Customer Price = Target Price × (1 - Discount%)	Simulation Hub
Base/New Price = Cost × 1.35 (fixed 35 % baseline margin)	Digital Twin
New Cost = Cost × (1+Steel%) × (1+Fuel%)	Digital Twin, Contract Analyzer
Current/Future Margin % = (Price - Cost)/Price × 100	Contract Analyzer, Digital Twin
Margin Loss = Current Margin - Future Margin	Contract Analyzer
Elasticity sweep: Target Price = Cost×(1+Margin%), Demand Factor = 1 - (Margin-20)×0.02, Expected Demand = max(int(Demand×Factor),1), Revenue/Profit from those	Price Engine — Run Price Optimization button
Price Gap % = (Our Price - Market Avg)/Market Avg × 100	Competitor Pricing
Competitive Health Score (fast path) = clamp(100 - gap×5 , 0, 100)	Competitor Pricing (Price War Sim, Position Matrix only)

12.2 Decision Logic (AI-Driven — judgment, not arithmetic)

Per repeated, explicit source comments, the following are **intentionally not formulas**: PSI margin %, capacity price modifier %, win probability %, discount-level win probability and “best scenario” selection, Executive Pricing Health Score, Margin Leakage health score, Advisor scenario-health score, Contract Risk score, and Competitor market-position health score. All of these are produced by Cortex AI reasoning over deterministic facts, via the shared `ai_reason_score` pipeline, specifically so that “no manual weights, averages, or formulas are used” for what the source repeatedly frames as judgment calls rather than calculations.

12.3 Alert Generation Logic

Executive Alerts and Competitor Alerts both follow the same two-step pattern: (1) a deterministic numeric threshold detects a **candidate** signal (e.g., “N combinations below 5 % margin”), and (2) AI_FILTER decides whether that candidate is significant enough to actually display, with AI_COMPLETE writing the final one-sentence alert text. Both call sites fail open.

13 Workflow Documentation

13.1 User Workflow (typical session)

1. Open the app → Overview tab loads automatically with headline KPIs.
2. (Optional) Open Cortex AI Diagnostics and run a connectivity test if AI cards are showing “AI Unavailable”.
3. Go to Price Engine → pick a SKU and customer → run Customer Price Lookup → review PSI-driven margin recommendation → review Capacity modifier for a chosen plant → review Win Probability → review Smart Discount Optimizer’s recommended discount.
4. Optionally use the independent “Run Price Optimization” elasticity sweep at the bottom of the same tab for a second, unrelated recommended-price figure.
5. Go to Simulation Hub for ad-hoc or pre-defined scenario comparisons.
6. Go to Digital Twin to model a future cost/demand shock, then to AI Advisor to get a narrative read-out of that same shock.
7. Go to Contract Analyzer to check a specific customer contract’s margin erosion risk.
8. Go to Competitor Pricing to benchmark against 4 manually-entered competitor prices, run the Price War slider, and view the (synthetic) historical trend.
9. Return to Executive Decision Center for the consolidated CEO/CFO-level view assembled from the above data.

13.2 AI Workflow (every scored call)

See “Request / Response Flow” in the Architecture chapter — Build Context → Retrieve Memory → Prompt → Parse → Coerce → Retry-once-on-failure → Cache → Log to Memory → Render.

13.3 Database Workflow

Nearly every tab issues its own independent `session.sql(...).to_pandas()` calls at render time (no shared repository/DAO layer, no query result caching via `st.cache_data`). Writes happen via three different mechanisms: (a) literal `INSERT INTO ... VALUES (...)` with f-string-interpolated values, (b) `session.create_dataframe(df).write.mode("append").save_as_table(...)`, and (c) a one-time `CREATE TABLE IF NOT EXISTS` for the AI memory log.

14 Security

14.1 Authentication & Authorization

There is **no in-application authentication or authorization code**. The app relies entirely on Snowflake's native Streamlit-in-Snowflake hosting model: a user must already be authenticated to Snowflake and hold a role with USAGE on the app before they can open it at all, and `get_active_session()` inherits that role's privileges for every query. There is no user-level feature gating inside the app itself (e.g., no check for "is this user allowed to see Executive Decision Center").

14.2 Data Validation

Widget-level validation is limited to Streamlit's own input constraints (`min_value`, slider ranges). There is no server-side re-validation of, for example, negative costs beyond a single `if cost <= 0` check in the price-optimization button handler.

14.3 SQL Injection Prevention

Every SQL statement is built with Python f-strings and manual escaping via `_sql_escape()`, which escapes backslashes and single quotes. **This is not parameterized-query binding.** For values drawn from `st.selectbox` (constrained to values already present in the table), the practical risk is low. For values that flow into AI_COMPLETE/AI_CLASSIFY/AI_FILTER/AI_AGG prompts — including user-typed competitor names in the Competitor Pricing tab — `_sql_escape()` is the only defense against a crafted string breaking out of the quoted literal. This is flagged as a **security-relevant limitation** in the Known Limitations chapter; the standard mitigation (Snowpark bind variables / `session.sql(query, params=[...])`) is not used anywhere in the file.

14.4 Caching & Session State

Caching is purely in `st.session_state` (see AI Documentation → Caching). No `st.cache_data/st.cache_resource` decorators are used anywhere in the file, meaning every non-AI Snowflake read re-executes on every rerun.

14.5 AI Safety

Every Cortex call is defensively wrapped: failures never raise past the wrapper, malformed JSON triggers one repair retry then a clearly labeled neutral result, and the UI always tells the user when a number is `ai_generated: False` rather than presenting it as if it were a real recommendation.

15 Performance

15.1 Caching

- AI results: session-scoped memoization by content hash (see above) — the single most impactful caching decision in the file, since it prevents repeat Cortex spend/latency for identical inputs within a session.
- Data reads: **no caching**. Every tab re-queries Snowflake on every Streamlit rerun (which happens on essentially every widget interaction anywhere on the page, per Streamlit's execution model), including SELECT * pulls with no row limit in the Overview and Executive Decision Center tabs.

15.2 Database Calls

Several tabs issue 3–6 separate `session.sql(...)` calls per render (Executive Decision Center alone loads 6 tables via `_safe_load`). None of these are batched into a single query or view.

15.3 Batching (AI)

Two explicit batching optimizations exist: (1) Top Opportunities / Customer Segmentation / Product Classification each request scoring for up to 8–10 rows in **one** `AI_COMPLETE` call rather than one call per row; (2) the Smart Discount Optimizer's 5 discount levels are scored in **one** `ai_reason_score_batch` call.

15.4 Scalability / Concurrency / Expected Load

No information is present in the source about concurrency handling, connection pooling, or expected load — Streamlit-in-Snowflake manages session isolation natively, but the app itself adds no additional scaling logic (no async, no background jobs, no queueing of Cortex calls).

15.5 Performance Recommendations

1. Add `st.cache_data(ttl=...)` around the repeated `_safe_load`/table-read calls, especially in Executive Decision Center and Overview.
2. Replace SELECT * with explicit column lists and row limits where full history isn't needed (e.g., `USER_SIMULATION_RESULTS`, `CUSTOMER_PRICING`).
3. Consider persisting the AI cache beyond session scope (e.g., a Snowflake results table keyed by prompt hash) so identical scoring requests across users/sessions don't re-incur Cortex latency/cost.

16 Error Handling

- **Pattern:** almost every data-access and AI-access code path is wrapped in try/except Exception, with a `st.info/st.warning/st.error` message and either an empty DataFrame or a clearly labeled neutral value substituted, so the tab keeps rendering rather than crashing.
- **Retries:** exactly one retry is implemented, and only for the central `ai_reason_score` JSON pipeline (a single “repair” prompt asking for JSON-only output if the first response didn’t parse).
- **Logging:** there is no structured application logger (no logging module usage). Diagnostics are surfaced purely through `st.session_state["ai_last_error"]`/`["ai_last_raw"]`/`["ai_call_s"]` visible via the Cortex AI Diagnostics expander.
- **Recovery:** the `CREATE TABLE IF NOT EXISTS` for `AI_MEMORY_LOG` and every `INSERT/save_as_table` write are wrapped so a missing-privilege or missing-table condition degrades to “memory not persisted this run” rather than a crash.

17 Testing

No test files, test framework imports (pytest, unittest), or CI configuration were supplied or found in the source. This is documented as a gap, not assumed to exist elsewhere.

17.1 Recommended Test Cases (not currently implemented)

- Unit tests for every deterministic formula in the Business Logic chapter (cost/margin/revenue/profit arithmetic), including boundary cases: zero cost, zero base value in `safe_pct_change`, negative discount, 0 % and 100 % discount.
- Unit tests for `parse_ai_json` against malformed LLM outputs: fenced JSON, JSON with leading prose, trailing commas, non-JSON text.
- Unit tests for `_coerce_score_payload` against missing/malformed fields (score as "85%", missing risks, risks as a bare string).
- Integration tests (mocking `session.sql`) confirming that every AI-failure path renders the documented "AI Unavailable" state rather than raising.
- Manual QA checklist: run each tab with (a) all tables populated normally, (b) one required table empty, (c) Cortex AI temporarily disabled via the model override, to confirm graceful degradation in all three states.

18 Deployment Guide

18.1 Prerequisites

- A Snowflake account with **Streamlit-in-Snowflake** enabled and Cortex AI functions (AI_COMPLETE, AI_CLASSIFY, AI_FILTER, AI_AGG, SNOWFLAKE.CORTEX.SUMMARIZE) available for the target region/edition.
- The base tables listed in the Database Documentation chapter must already exist under PRICING_ENGINE_DB.CORE_INPUT / CORE_OUTPUT / CORE_INTERNAL (this file contains no DDL to create them).
- The running role needs USAGE on SNOWFLAKE.CORTEX_USER (per the in-app diagnostics guidance) and CREATE TABLE on PRICING_ENGINE_DB.CORE_OUTPUT (for the best-effort AI memory log table; the app functions without this privilege, just without persisted memory).

18.2 Environment Setup

Because this runs as Streamlit-in-Snowflake, there is no separate Python environment to provision — Snowflake manages the runtime. No `requirements.txt` was supplied in this codebase; any third-party Python packages beyond what Snowflake's SiS runtime provides by default would need to be declared in the app's package list (not present in the file reviewed).

18.3 Configuration

No `st.secrets`, environment variables, or config files are referenced anywhere in the source — the only “configuration” surface is the in-app Cortex model selector, which is a runtime UI control, not a deployment-time setting.

18.4 Execution / Production Deployment

Deploy as a Streamlit-in-Snowflake app pointing at this file as the main module. No additional deployment steps (Docker, CI/CD pipeline, infrastructure-as-code) are present in the source to document.

19 Maintenance Guide

19.1 Coding Standards Observed

- Consistent section-banner comments (# —...—) delimiting logical regions.
- Consistent `_init_state(key, default)` guard pattern for every persisted session key.
- Consistent “deterministic math stays in Python; judgment goes to Cortex” principle, restated in comments at nearly every AI integration point.

19.2 Extension Guide

To add a new AI-scored module: call `ai_reason_score(module="<new_module>", decision_type="<decision>", context_facts="<deterministic facts as text>", extra_instruction="<what the AI should weigh>")`, then render with `render_ai_score_card(payload)`. This reuses history, caching, retry, and fallback behavior automatically.

19.3 Best Practices Going Forward

- Remove or clearly quarantine the superseded v1 helper functions identified in the Function Documentation chapter, to avoid future contributors calling the wrong (fixed-rule) code path by mistake.
- Reconcile the two independent pricing models on the Price Engine tab (AI chain vs. elasticity-sweep optimizer) and the fixed 35 % baseline margin in Digital Twin, so a single SKU does not show three different “recommended price” philosophies across tabs.
- Introduce `st.cache_data` for read-only table pulls.

20 Known Limitations

1. **Single-file monolith** — no module separation, no shared repository/DAO layer, no tests, no CI, as supplied.
2. **Two (or three) independent, unreconciled pricing models** live in the same app: the AI-driven PSI→Capacity→WinProb→Discount chain, the deterministic elasticity-sweep optimizer, and Digital Twin's fixed 35 % baseline margin.
3. **Synthetic historical trend data** in Competitor Pricing's Historical Trends sub-tab — not real time-series data, generated by multiplying current prices by a hardcoded ratio array.
4. **Dead/superseded functions retained in source** (`get_capacity_modifier`, `get_capacity_recommendation`, `ai_capacity_recommendation`, `get_psi_margin`, `get_psi_recommendation`, `get_psi_score`, `get_psi_score_label`, `ai_psi_recommendation`, `ai_win_probability_explanation`) — none of these are invoked by the live UI paths documented above.
5. **DYNAMIC_SURCHARGES table is loaded but never used** in Executive Decision Center.
6. **No parameterized SQL** — manual string escaping only (`_sql_escape`), not Snowpark bind variables.
7. **No `st.cache_data`** on any table read — every rerun re-queries Snowflake.
8. **No automated tests** were found or supplied.
9. **Possible column-naming inconsistencies** between tabs for the same conceptual table (e.g., `PSI_SCORE` vs. `PRICE_SENSITIVITY_INDEX`, `UTILIZATION_PCT` vs. `CURRENT_UTILIZATION`) — both are documented as separate **[Inferred from usage]** assumptions because the two tabs' queries don't share identical column names for what appears to be the same underlying concept.
10. **Competitor Pricing tab does not read `COMPETITOR_PRICING` table** — all 4 competitor prices are manually re-entered by the user every session; only the Win Probability Predictor (a different tab) reads that table.
11. **No authentication/authorization logic inside the app** — fully dependent on the hosting Snowflake role.

21 Conclusion

The Pricing Intelligence Platform is a functionally rich, single-file Streamlit-in-Snowflake application that pairs deterministic manufacturing pricing arithmetic with a consistently-applied Snowflake Cortex AI reasoning layer for every judgment call (risk, segmentation, narrative, recommendation). Its central `ai_reason_score` pipeline is a genuinely reusable piece of architecture, applied uniformly across nine scoring modules with real defensive engineering (caching, retry-once, explicit “AI Unavailable” states, fail-open alerting). Its main structural weaknesses — an in-place v1→v2 migration that left dead code behind, unreconciled parallel pricing models, one synthetic-data visualization, and the absence of tests/caching/parameterized SQL — are documented above and carried forward as the Technical Debt and Roadmap sections of the companion Feature & Roadmap document.

22 Appendix

22.1 Glossary

- **PSI** — Price Sensitivity Index, a per-customer 0–1 score (0 = insensitive, 1 = highly sensitive) used to drive margin strategy.
- **CUM** — Capacity Utilization Modifier, a price adjustment (in this app now AI-judged, historically a fixed lookup) based on plant utilization %.
- **SiS** — Streamlit-in-Snowflake, the native hosting model this app is built for.
- **Cortex AI** — Snowflake’s built-in LLM function family (AI_COMPLETE, AI_CLASSIFY, AI_FILTER, AI_AGG, SNOWFLAKE.CORTEX.SUMMARIZE).
- **ai_reason_score** — the app’s central reusable AI decision pipeline.
- **Fail-open** — a design choice where, if an AI significance check can’t be completed, the system defaults to showing the information rather than hiding it.

22.2 Abbreviations

SKU (Stock Keeping Unit) · KPI (Key Performance Indicator) · CFO/CEO · SQL · DDL · JSON · LLM · SiS.

22.3 References

Single source file supplied: `streamlit_app.py` (the “Pricing Intelligence Platform”), analyzed in full for this document.